RYDA · DAILY-RENTAL MARKETPLACE

RYDA Daily-Rental Marketplace — Build Loop

Engineering build guide · prepared 11 August 2026

> **What this document is.** A step-by-step build guide that a Claude Code session runs **in a loop** to evolve RYDA from a rental *lead-gen* site into a full **ZocDoc-style daily exotic-car rental marketplace**: rental companies (operators) list cars, renters book them by the day against a per-car availability calendar, the operator approves each request, and the money moves **on-platform** with a configurable booking fee.
>
> **For the founder (non-technical):** you do not need to read the phases. Hand this file to a Claude Code session working in the RYDA repo and say "*work the RYDA_RENTAL_BUILD_LOOP, one task at a time.*" It tells Claude exactly what to build, in what order, how to stay safe in a shared codebase, and how to prove each step works before moving on. The **Decisions already made** and **Open defaults** sections are the only parts written for you — skim those and change anything you disagree with.
>
> **Source of reusable patterns:** the sibling Mainstable / eqmarket marketplace (`C:\dev\mainstable-app`) has already solved most of this (auth, verification, threaded messaging, notifications, a Stripe Connect rail, and a **database-enforced slot-booking state machine**). Where a phase says "*donate from Mainstable,*" it means copy the **pattern / data-model / logic** and re-skin it into RYDA's own design system — **never** copy Mainstable's CSS or its `@supabase/ssr` plumbing (RYDA uses the raw client).

0. The loop protocol — how to work this document

Work **one task at a time**. A "task" is a single checkbox in a phase (§4). Never batch a whole phase into one PR.

Each iteration of the loop:

1. **Orient.** Re-read this file's guardrails (§3) and the Decisions (§2). Open the current phase. Read the *real* RYDA files it names before writing anything — the codebase is the ground truth; this guide can drift.
2. **Sync & safety-check.**

```
   git fetch origin
   git status --porcelain          # MUST be empty. If not → STOP (see §3).
   git branch --show-current       # base branch must be feat/dt-rental-first-site
   git pull --rebase origin main   # keep current; the rental site lives on the branch, main is co-
ownership
```

If `git status` shows changes you did not make, **stop and ask a human** — another agent may be mid-task in this shared tree.

3. **Branch.** Cut a small task branch off the rental branch:

```
git checkout feat/dt-rental-first-site
git checkout -b feat/dt-<slug>      # <slug> = the task, e.g. rental-listings-table
```

4. **Claim your surface out loud.** If the task touches a **shared surface** (`site-header`, `/admin/*`, `auth`/`signup`, design tokens in `globals.css`, `src/lib/fees.ts`, or **any new DB migration**), the PR **title** must say what you are claiming, e.g. `feat(rental): rental_listings table — claims migration 00NN`. For migrations, `ls supabase/migrations | tail` first and claim the **real** next number (see §3).
5. **Implement the smallest next task.** Match RYDA conventions (§1). New money logic in `fees.ts` ships with Vitest unit tests in the same PR (precedent: `src/lib/__tests__/rental-fees.test.ts`).
6. **Verify — do not push red.**

```
npm run verify        # = tsc --noEmit && vitest run && next build
npm run test:e2e      # only for browser-facing changes (Playwright; not in verify)
```

Then run the **phase's acceptance check** (each phase lists one). Both must pass.

7. **Open a small PR** against `feat/dt-rental-first-site` (**never** `main`). Keep it under ~400 lines where possible. Wait for `verify` green before it merges.
8. **Record progress.** Tick the checkbox in §7 (or append to a `docs/RENTAL_BUILD_LOG.md`) with the PR link and one line on what changed.
9. **Repeat** from step 1 with the next task.

Migrations are proposed, not applied. You may *write* a migration file, but **applying it to any database requires explicit operator approval** (see §3). Do not run `supabase db push` or paste SQL into the Supabase editor on your own.

---

1. Load this context before writing any code

Read these first (they are the RYDA conventions every task must obey):

- **`AGENTS.md`** (repo root) — the collaboration contract. Non-negotiable.
- **`.claude/skills/frontend-design/SKILL.md`** + **`src/app/globals.css`** — the design system. Tailwind v4, **light-only**, tokens only (`bg-cream`, `text-ink`, `text-ink-soft`, `text-mute`, `border-rule`, `bg-red`/`bg-red-deep`, `text-marine`, `bg-success`…). **No raw hex in components.** Fonts: Fraunces = `font-display`, Inter = `font-sans`. **Cars use the `red` accent, boats use `marine` — never mix.** The dense browse-card pattern in `src/components/rental-listings.tsx` is canonical for the rental grid.
- **`src/lib/fees.ts`** — the *only* home for money math. Holds `computeFees` (co-ownership, **dollars**) and `computeRentalFee` (rental, **cents**). You extend `computeRentalFee`; you never touch `computeFees` or the `*100` dollars↔cents boundary.
- **`src/lib/stripe.ts`** — server Stripe client, API version pinned; three webhook-secret constants.

- `src/lib/supabase-admin.ts` (service-role, `server-only`, bypasses RLS — what ~63 API routes use), `src/lib/supabase.ts` (browser anon), `src/lib/api-auth.ts` (`getUserFromRequest`), `src/lib/admin-auth.ts` (`requireAdmin`, role from `app_metadata`). **There is no `@supabase/ssr` and no `middleware.ts`** — do not introduce the SSR helper; follow the existing patterns.
- The current rental funnel, to understand what you are replacing: `src/components/rental-inquiry-form.tsx`, `src/app/api/rental-inquiry/*`, `src/lib/partner-fleet.ts`, `src/lib/market-data.ts`, `src/components/rental-listings.tsx`, `src/app/rent/*`, `src/app/cars/*`.
- The Stripe rail you are inverting: `src/app/api/admin/inquiries/[id]/payment-link/route.ts`, `src/app/api/stripe/connect-webhook/route.ts`, and the on-platform reference already in the repo: `src/app/api/share-purchase/**` (co-ownership charges on the *platform* account — the model for the new rental rail, including its refund route).
- The booking primitive to reuse: `supabase/migrations/0021_bookings_no_double_book.sql` (the `btree_gist` EXCLUDE constraint). The co-ownership handover flow to rewire: `0034_vehicle_handovers.sql` + `src/app/bookings/[id]/checkin` + `/return` + `src/app/api/bookings/[id]/handover/route.ts`.
- `docs/DROPBOX_SIGN_SETUP.md` — Dropbox Sign is already installed (`@dropbox/sign`) and configured; it is the tool for the reservation agreement (Phase 4).

Runtime facts: Node **24** is mandatory (`.npmrc` `engine-strict=true`; wrong major fails `npm install`). Migration **head present = `0043`; the next number is almost certainly `0044` — but always re-check** (`ls supabase/migrations | tail`) because other agents add migrations too. CSP in `vercel.json` is currently **Report-Only**; `next.config.ts` `images.remotePatterns` only allows Unsplash/Wix — both must be extended when operator car photos and Stripe on-platform surfaces arrive.

---

2. Decisions already made (founder-approved — treat as settled)

#	Decision	What it means for the build
D1	**Full payment on-platform**	The rental total is charged on **RYDA's** Stripe account; RYDA's booking fee is retained; the remainder is transferred to the operator. Money is no longer a direct charge to the operator.
D2	**Configurable fee engine**	Per-operator, **percent OR flat**, and **either payer** (renter-pays-on-top or operator-pays-out-of-payout). Optional floor/cap. Extend `computeRentalFee`; add columns to `partners`.
D3	**Request-to-book (operator approves)**	Renter requests specific dates → operator **approves / declines / proposes alternates** → booking confirms. Not instant-book. Auto-expire stale requests.

#	Decision	What it means for the build
D4	**Payout after clean return**	Operator is paid **after** the car is returned in good condition. Implemented as **separate charges + transfers** (charge renter on approval, hold funds on platform, transfer to operator on completion).
D5	**Security deposit = card hold**	An authorization-and-hold (Stripe **manual-capture** PaymentIntent) placed at approval; **nothing captured** unless the operator files a post-return claim. Released on clean return.
D6	**Operator revealed after confirmation**	Operators stay anonymous ("a vetted Miami operator") while browsing/requesting; the operator identity is revealed to the **renter only after the booking is confirmed**, and in-app messaging opens then.
D7	**Partner operators only (for now)**	Every listing belongs to a partner with their own Connect Express payout account. A "RYDA rents its own fleet" path is explicitly deferred.

The money flow these decisions produce (the reference sequence):

1. **Request** — renter selects an available date range → submits a request. A `rental_bookings` row is created `status = requested`, with a **frozen quote snapshot** (base = daily rate × billable days ± discounts, plus the fee lines per D2). A Stripe **SetupIntent** saves/validates the renter's card. **No money moves.** `requested` does **not** reserve the dates.
2. **Operator decision** — approve / decline / propose. Default **auto-expiry 24h** → auto-decline.
3. **On approve** — (a) **charge the rental total** off-session on the platform account (funds land in RYDA's balance, **not** transferred yet); (b) place the **deposit hold** (separate `capture_method:'manual'` PaymentIntent, authorized only); (c) move `rental_bookings → confirmed` — this transition is what the **no-double-book EXCLUDE constraint** guards, so a same-date race loses here and is auto-refunded/voided; (d) **reveal the operator** to the renter and open the messaging thread; (e) issue the **reservation agreement** (Dropbox Sign).
4. **Pickup** — check-in via the (rewired) `vehicle_handovers` flow → `in_progress`.
5. **Return** — operator confirms clean return → `completed`: **release the deposit hold** (or capture up to the held amount on a claim), then **transfer the operator's net** to their Connect account (gated on `payouts_enabled && details_submitted`).
6. **Cancellation / refund** — per the policy tiers (§5, O3); reverse the deposit hold; the fee-refund rule decides who eats the booking fee.

> **Deposit-hold caveat to encode:** Stripe card authorizations expire (~7 days, up to ~31 with extended auth on eligible cards). For rentals longer than the hold window, either re-authorize before expiry (a scheduled job) or fall back to charge-and-refund for that booking. Handle short rentals first; flag long rentals as a known edge case.

---

3. Guardrails — the rules the loop must never break

These come straight from `AGENTS.md` and the RYDA inventory. Breaking one is worse than being slow.

1. **Never commit or push to `main`.** Rentals live on `feat/dt-rental-first-site`; branch off it. A `.githooks/pre-push` refuses `main` (emergency override `RYDA_ALLOW_MAIN_PUSH=1` — do not use it) and CI's `direct-push-guard` flags any bypass.
2. **Small PRs, `npm run verify` green before every PR.** `verify = tsc --noEmit && vitest run && next build`. E2E (`npm run test:e2e`) for browser changes. **There is no lint step** despite the README — do not rely on it.
3. **Migration numbers are the collision hazard.** Multiple agents share this tree. `git fetch origin && ls supabase/migrations | tail`, claim the **real** next number in the PR title, **never renumber an applied migration**. **Applying** a migration needs **explicit operator approval** — you write the file, a human runs it (SETUP.md §1).
4. **Node 24 only.** Wrong major fails `npm install` by design.
5. **Co-ownership must keep working and stay out of rental surfaces.** Do **not** delete or touch `/portfolio`, `/membership`, `/co-ownership`, the boats tree, `share-purchase`, `share_holdings`, `llc_*`, or the co-own `bookings` (0009/0021). Do **not** pull any of it into rental nav.
6. **Never reuse co-ownership tables for rentals.** The co-own `bookings`, `llc_messages`, and `is_llc_member()` are share-entitlement-scoped — reusing them entangles logic and is a **security regression** on messaging. Build **new** rental tables every time (`rental_listings`, `rental_availability`, `rental_bookings`, rental messages…).
7. **RLS posture is deliberate — set it per-table, don't relax existing policies.** - `rental_listings`, `rental_availability` → **publicly readable** (renters browse open days pre-auth). This is the **opposite** of the co-own calendar (members-only). Add these as **new** policies; do not touch the co-own policy. - `rental_bookings` → renter-scoped SELECT (a renter sees only their own). - `partners`, `rental_payments` → **service-role only, zero client policies** (commission rates and `acct_` ids must never reach the browser). - Keep `import "server-only"` on `supabase-admin.ts` / `stripe.ts`. The service-role key and Stripe secret are client-poison. New money/booking routes use the service-role client and **enforce ownership in code** (RYDA's dominant pattern), or the inline user-scoped client pattern seen in `src/app/api/account/votes/[id]/route.ts`.
8. **Enforce state at the database, not just the route.** Donate Mainstable's approach: a **money-guard trigger** (`transactions_money_guard`-style) that allows only legal status transitions with service-role-only doors, and a slot state-machine trigger (write-once, terminal-stays-terminal). Extend the existing `rental_payments_enforce_status` immutability trigger — do not fight it.
9. **The payment copy rule inverts on the pivot — sweep it in the same PR as the rail.** Today the code says "*RYDA never holds your money.*" After D1 that is **false**. When Phase 3B lands, the same PR must correct every such claim. Known locations to fix: `AGENTS.md` (the doctrine paragraph, ~lines 27–30), `src/app/how-it-works/page.tsx` (~118), `src/app/rent/[symbol]/page.tsx` (~407), `src/app/rent/booking-confirmed/page.tsx` (~53), `src/app/partners/page.tsx` (~72), `src/app/member-protection/page.tsx` (~41), admin copy in `src/app/admin/inquiries/page.tsx` (~746) and `src/app/admin/partners/page.tsx`

(~29), the customer email in `payment-link/route.ts` (~147), and the doctrine comments in `payment-link/route.ts` (1–13), `connect-webhook` header, `onboarding-link/route.ts` (11–15), and `0041`'s header. **State what the code does — never over-promise holds/guarantees.**

10. **Operators are never named to customers before confirmation** (D6). The current boundary is real: the `replyTo` indirection and the `GET /api/rental-inquiry` stripping `partner_name`. Preserve anonymity through browse + request; reveal only post-confirm.
11. **Kill the `partners.name` string coupling before self-serve fleet.** `partner-fleet.ts`, `rental_inquiries.partner_name`, and the pay-link operator resolution all join on the operator **name** string; renaming an operator breaks in-flight leads. Migrate to a `partner_id` FK (Phase 0D) **before** operators manage their own listings.

---

4. The build phases

Each task carries: **Goal**, the **RYDA files/migrations** to touch, the **Mainstable pattern** to borrow (if any), a **Definition of Done (DoD)**, and an **Acceptance check** (what to run/observe to prove it — in addition to `npm run verify`).

> Ordering law: **listings table → calendar/booking → payment rail → copy sweep.** Parallelizable side-tracks: notifications, fee engine, role model, identity verification, media bucket, messaging (once the schema lands).

Phase 0 — Foundation *(the keystone; blocks Phases 2–3)*

☐ **0A. `rental_listings` table + media bucket + operator-scoped RLS.** *(keystone)*

Files/migrations: new `00NN_rental_listings.sql`; a Supabase **Storage** bucket `rental-car-photos` (public read, per-operator-folder write RLS); begin the DB-backed data path that will replace `src/lib/market-data.ts` / `src/lib/partner-fleet.ts` for rentals.

Pattern (Mainstable): the generic `listings` spine + the **asset "passport"** concept (Mainstable keys a permanent asset that persists across listings/owners → here a **VIN-keyed car**) + the **public bucket + folder-per-owner Storage RLS**. Columns: `id`, `partner_id → partners(id)` (FK, **not** the name string), `vin`, `make`, `model`, `year`, `market`, `daily_rate_cents`, `min_nights`, `max_nights`, `status` (`draft|active|paused|archived`), `hero_photo_path`, `created_at`. Photos in a child `rental_listing_photos` table (first = cover), mirroring Mainstable's `listing_photos`.

RLS: public SELECT on `active`; operator INSERT/UPDATE scoped to their `partner_id`.

DoD: an operator row can own listings; photos upload to the bucket; `/rent` can read from the DB (behind a flag or new server path) without breaking the existing static grid.

Acceptance: seed one listing via a script/SQL, load `/rent`, see it render from the DB; confirm an anonymous client can read it and a non-owner operator cannot mutate it.

☐ **0B. First-class `renter` and `operator-staff` roles.**

Files: `src/lib/admin-auth.ts` (extend the role model), signup/onboarding, route gates.

Pattern (Mainstable): the multi-role concept + a DAL-style `getUserRoles` / `requireRole` helper — **re-implemented on RYDA's raw client + route-level gates** (keep admin's `app_metadata` trust model; do **not** import `@supabase/ssr`).

DoD: a user can be a renter and/or an operator-staff member; routes gate on role.

Acceptance: unit test the role helper; a renter cannot reach operator-only routes.

- [ ] **0C. In-app notifications table + `notify()` helper.**

Files: new `00NN_notifications.sql`; `src/lib/notify.ts` (extend beyond email); a notifications feed under `/account`.

Pattern (Mainstable): `notifications(user_id, type, title, link, read)` + `notify()` + an admin fan-out RPC. RYDA has email only today.

DoD: server code can drop an in-app notification; the renter/operator sees a feed.

Acceptance: trigger a test notification; it appears in the feed and marks read.

- [ ] **0D. Migrate `partners.name` string coupling → `partner_id` FK.** *(do before 2/3)*

Files: new `00NN_partner_id_fk.sql`; `src/lib/partner-fleet.ts`, `rental_inquiries.partner_name` usage, `payment-link` operator resolution.

DoD: everything joins on `partner_id`; renaming an operator no longer breaks leads.

Acceptance: rename a test operator; existing inquiries still resolve; pay-link/booking path still finds the operator.

Phase 1 — Renter identity & onboarding *(parallel with Phase 0 tail)*

- [ ] **1A. Renter identity + driver's-license & age verification.** *(blocks self-serve booking)*

Files: `src/app/api/kyc/*` (extend the existing scaffold to a rental gate), `src/lib/kyc-verified-outputs.ts`, re-enable the disabled Checkr/license cards in `src/app/account/verification/page.tsx`, persist the onboarding "Driver's license #". Add a **separate rental age constant (default 25)** — do **not** reuse `src/lib/age.ts`'s co-owner **28**.

Pattern (Mainstable): the KYC scaffold (owner submits → admin approves → `is_verified()` drives a public **Verified badge**), designed to swap in a real IDV provider. Stripe Identity is already integrated; add Checkr for driving record (see §5, O7).

DoD: a renter must pass license + age verification **before** a booking can confirm.

Acceptance: an unverified renter is blocked at request→confirm; a verified one proceeds.

- [ ] **1B. Rental-flavored signup / onboarding copy.** Fork the co-own onboarding wizard for the renter path; keep co-own onboarding intact.

Acceptance: a new user can sign up as a renter and reach an empty "My rentals" surface.

Phase 2 — Availability calendar + booking *(the centerpiece; BLOCKED by 0A)*

- [ ] **2A. Per-car availability + blackout model.**

Files: new `00NN_rental_availability.sql`. Default-open operating window per listing, plus explicit operator **blackout ranges** (maintenance / personal / off-platform bookings). Public SELECT (renters see open days).

Acceptance: an operator blackout removes those days from the car's public calendar.

- [ ] **2B. `rental_bookings` table + no-double-book EXCLUDE + state-machine trigger.**

Files: new `00NN_rental_bookings.sql`. Columns: `id`, `listing_id`, `renter_user_id`, `start_date`, `end_date`, `status`, and the

frozen quote snapshot (`base_amount_cents`, `fee_cents`, `fee_payer`, `deposit_amount_cents`, `renter_total_cents`, `operator_net_cents`), `client_token` (idempotency — borrow the pattern from `0039_rental_inquiries`).

Copy the 0021 primitive verbatim (`btree_gist` is already enabled):

```
ALTER TABLE rental_bookings ADD CONSTRAINT rental_bookings_no_overlap
  EXCLUDE USING gist (
    listing_id WITH =,
    daterange(start_date, end_date, '[]') WITH &&
  ) WHERE (status IN ('confirmed','paid','in_progress'));
```

So `requested` / `pending` do **not** reserve; the reservation fires on the transition to `confirmed` (§2 step 3c), and a same-date race loses cleanly there. **Pattern (Mainstable):** the trigger-driven slot state machine (`requested → confirmed → paid → in_progress → completed`; write-once; terminal stays terminal; only the operator confirms). Enforce transitions in a **trigger**, not just the route. **RLS:** renter-scoped SELECT; writes via service-role routes.
Acceptance: two overlapping confirmations on the same car — the second fails at the DB; an illegal status jump (e.g. `requested → completed`) is rejected by the trigger.

- [ ] **2C. Availability-aware calendar selection UI + server quote calculator.**

Files: extend `src/components/asset-calendar.tsx` (month-grid shell, `grid-cols-7`, month nav — currently display-only) into **availability-aware date-range selection**; replace the raw `<input type="date">` pairs in `src/components/rental-inquiry-form.tsx` and `src/app/bookings/new/page.tsx`; give `src/app/rent/*` a **server data path** (the existing static prerender must gain a dynamic branch). A **server-side quote** computes `dailyRate × billable nights ± discounts` — no client-trusted pricing.

DoD: a renter picks a valid range on a car's calendar and sees an accurate quote.

Acceptance (E2E): select dates → the quote matches the server calculation; blacked-out and already-booked days are unselectable.

- [ ] **2D. Request-to-book flow (request → approve/decline/propose → confirm).**

Files: new routes under `src/app/api/rental-bookings/*` (request, operator-decision); operator surface in the fleet dashboard (see 2F); renter surface under `/account`. Implements §2 steps 1–3 **except** the charge (that lands with Phase 3B — until then, gate behind Stripe **scaffold mode** so the flow is walkable without live charges). Default

auto-expiry 24h (see §5, O-expiry).

Acceptance: operator approves → booking `confirmed`, dates lock, operator revealed to the renter; decline/expiry frees the dates and notifies the renter.

- [ ] **2E. Renter ↔ operator threaded messaging (booking-scoped).**

Files: new `00NN_rental_messages.sql` (**do not** reuse `llc_messages`); a thread keyed by `rental_booking_id`; reuse the 30s-polling UI pattern from `src/app/messages/*`.

Pattern (Mainstable): the `(listing, buyer, seller)` thread model; hide last names until the deal is live.

Honor D6: the thread opens (and the operator is named) **only after confirmation**.

Acceptance: pre-confirmation, no operator identity/thread; post-confirmation, both parties can message in-app.

- [ ] **2F. Operator fleet dashboard — self-serve listings, pricing, availability, requests.**

Files: flesh out the empty-state `FleetPanel` in `src/app/partner/page.tsx`; `src/app/api/partner/me/route.ts`.

DoD: an approved operator can add/edit a car, set daily rate + min/max nights, manage availability/blackouts, and approve/decline/propose booking requests.

Acceptance: end-to-end as an operator: list a car → it appears on `/rent` → receive and approve a request.

- [ ] **2G. Renter dashboard — "My rentals."**

Files: replace the read-only inquiry chips in `src/app/account/requests/page.tsx` with a real booking history (upcoming / active / past) sourced from `rental_bookings`.

Acceptance: a renter sees their requests and confirmed bookings with status.

Phase 3 — Payment pivot *(BLOCKED by the Phase 2 booking entity)*

- [ ] **3A. Configurable fee engine.** *(parallel to Phase 2; must merge with/before 3B)*

Files: extend `computeRentalFee` in `src/lib/fees.ts` (do **not** touch `computeFees` or the `*100` boundary); new `00NN_partner_fee_config.sql`. Generalize to a config object: `{ mode:'percent'|'flat', rate | flatCents, payer:'operator'|'renter', floor?, cap? }` returning `{ feeCents, renterTotalCents, operatorNetCents, applicationFeeCents }`.

- `payer:'renter'` → fee is **added on top** (renter total = base + fee; changes the amount charged, not just the split).
- `payer:'operator'` → fee is **deducted** from the operator's payout (today's behavior). Add `partners.fee_mode`, `partners.fee_flat_cents`, `partners.fee_payer` (keep `commission_rate` as the percent). **Widen the `[0,0.5]` contract in all three places that hard-code it:** (1) the `0041` DB CHECK, (2) the `computeRentalFee` guard, (3) the `src/app/api/admin/partners/route.ts` POST validation (~line 319).

  Ships with Vitest tests covering both payers × percent/flat × floor/cap.

  Admin UI: add mode/flat/payer controls to the `/admin/partners` Operators tab (only a commission-% editor exists today).

  Acceptance: the admin preview and the server charge call the **same** function and agree to the cent for every payer/mode combination (the exact divergence the file header warns of).

- [ ] **3B. Full on-platform rail — separate charges + transfers (D1, D4).**

Files: the booking approval route (from 2D) creates a **platform-account** charge — model it on `src/app/api/share-purchase/**` (RYDA's existing on-platform reference), **not** the connected-account direct charge. Because the charge now fires on the **platform** account, `connect-webhook/route.ts`'s `event.account == partner.stripe_account_id` cross-check (~lines 249–274) would **reject** it — **relocate** the rental handler to a platform-account webhook (its own `stripe_events.endpoint` tag so it doesn't poison the

`endpoint='connect'` dedup scope). On **completion** (return confirmed), create a **Transfer** to the operator's connected account, gated on `payouts_enabled && details_submitted`.

Migration: `00NN` on `rental_payments` — add `base_amount_cents`, `fee_payer`, gross/net/**transfer**/payout columns; **extend** `rental_payments_enforce_status` to add `refunded` / `disputed` states (a naive UPDATE currently `raise exception`s). Keep "paid is terminal; a re-quote is a **new row**."

Pattern (Mainstable): the **money-guard trigger** (every legal transition enforced at the DB, service-role-only doors) + payout gating + reconcile/reminder crons.

Acceptance (scaffold + test-mode): approve a booking → renter charged on the platform account → funds held → mark return → transfer to operator; each step is a legal, trigger- approved transition and the ledger balances.

- [ ] **3C. Security-deposit hold (D5).**

Files: at approval, create a **separate** `capture_method:'manual'` PaymentIntent for the deposit (authorized, not captured); on clean return, **cancel** it to release the hold; on a claim, **capture** up to the held amount. Record deposit state on `rental_bookings`. Encode the **auth-expiry caveat** (§2) — re-authorize long rentals or fall back to charge-and-refund.

Acceptance: approval places a hold visible in Stripe; clean return releases it; a claim captures the claimed amount and no more.

- [ ] **3D. Refunds, disputes, cancellation policy.**

Files: a rental refund route copied from `src/app/api/share-purchase/[id]/refund/route.ts` (CAS-before-refund, idempotency key, dispute gate `src/lib/dispute-status.ts`); on destination/transfer charges decide `refund_application_fee` / `reverse_transfer` (who eats the fee — see §5, O3). A rental dispute table linked to `rental_payments` + `charge.dispute.*` webhook handlers + admin actions (the existing `dispute_cases` / `/admin/disputes` are wired only to `share_purchases` and are read-only).

Acceptance: a renter cancellation within each policy tier refunds the correct amount and releases the deposit; an operator cancellation fully refunds; a Stripe dispute surfaces in admin.

Phase 4 — Polish & truthful launch *(after the rail is truthful)*

- [ ] **4A. Payment copy + legal sweep.** *(MUST land with or immediately after 3B)* Correct every claim listed in guardrail §3.9 to match D1. State what the code does.

Acceptance: grep the repo for "never hold" / "never touch" / "your price is the operator's price" style claims — none remain that contradict the on-platform rail.

- [ ] **4B. Reservation agreement / e-sign.** Use **Dropbox Sign** (already installed; see `docs/DROPBOX_SIGN_SETUP.md`) for a renter+ operator rental agreement issued at confirmation; or Mainstable's lighter **click-accept** (`/trial-agreement`) pattern if a full e-sign is overkill for v1.

Acceptance: a confirmed booking produces a signed/accepted agreement stored against it.

- [ ] **4C. Rewire check-in / return to the rental booking.**

Files: `0034_vehicle_handovers.sql`, `src/app/bookings/[id]/checkin` + `/return`, `src/app/api/bookings/[id]/handover/route.ts`. The flow is clean but its RLS ties reads to the co-own

`bookings.user_id`; rewire to `rental_bookings` + the renter. Pickup → `in_progress`; return → `completed` (triggers payout + deposit release, Phase 3).

Acceptance: a renter completes check-in and return against a rental booking; completion drives payout + deposit release.

☐ **4D. CSP enforcement + image allowlist + booking-expiry job.** Extend `next.config.ts` `images.remotePatterns` and the `vercel.json` CSP `img-src` for operator car photos and any Stripe on-platform surfaces, then flip CSP from Report-Only to enforced. Add booking-request auto-expiry — **note the Vercel Hobby daily-cron cap** (`docs/VERCEL_CRONS.md`): prefer a **lazy-expire on read** plus one daily sweep rather than a sub-daily cron.

Acceptance: operator photos load without CSP violations; stale requests expire.

---

5. Open defaults (I chose sensible ones — change any you disagree with)

These were **not** blocking, so the guide runs with the defaults below. Each is a `FOUNDER-DECISION` the loop should treat as settled unless you say otherwise.

#	Question	Default chosen	Change it if…
O1	**Min / max rental length**	Per-car, set by the operator (`min_nights` / `max_nights`); platform floor 1 night, ceiling 30	you want a platform-wide minimum (e.g. 3 nights)
O2	**Who sets per-car pricing**	**Operator self-serve** daily rate; optional weekend/seasonal surcharge + duration discount later	you want admin-set pricing at launch
O3	**Cancellation & fee-on-refund**	Tiered: free >7 days out; 50% 2–7 days; non-refundable <48h. **Renter cancellation → RYDA keeps its booking fee** (`refund_application_fee:false`); **operator cancellation → full refund incl. fee**	you want a different tier or fee treatment
O4	**Insurance / merchant-of-record**	Operator provides insurance; on-platform payment makes **RYDA merchant of record** for the charge — **flag to counsel** (refunds, chargebacks, the "$1M / 100 mi/day" promise)	counsel advises a different posture
O5	**Request auto-expiry**	**24 hours**, then auto-decline + notify	operators need longer to respond
O6	**KYC provider + age floor**	Extend **Stripe Identity** (already integrated) for ID + **Checkr** for driving record; rental **age floor 25**	your insurance requires a different age/provider

#	Question	Default chosen	Change it if…
O7	**Operator staff accounts**	One `partner_accounts` row = one login for v1	operators need multi-user teams now

> **Counsel items (do not decide in code):** O4 (merchant-of-record / insurance liability), the rental agreement terms (4B), and the cancellation/refund terms (O3) are legal decisions — surface them, don't settle them.

---

6. What already exists in RYDA (so you modify, don't duplicate)

Condensed from a full read-only inventory. **KEEP** = leave as-is; **MODIFY** = extend; **BUILD** = new; **DONATE** = port the Mainstable pattern.

Capability	RYDA today	Action
Auth / signup	BUILT (GoTrue; raw `@supabase/supabase-js`)	KEEP + light MODIFY (renter role, copy)
Account hub	BUILT, co-own-flavored (`src/app/account/*`)	MODIFY (separate rental sections)
Renter identity / driver verification	STUB (KYC scaffold wired only to co-own 28+ gate)	MODIFY + BUILD (1A)
Renter dashboard	PARTIAL (inquiry chips, no bookings)	MODIFY/BUILD (2G)
Operator / fleet dashboard	PARTIAL (account+Connect onboarding done; `FleetPanel` empty)	MODIFY + BUILD (2F)
Admin dashboard	BUILT, co-own-centric (`src/app/admin/*`, `requireAdmin`)	KEEP + MODIFY (rental payments/refunds/disputes)
Car listings + media	MISSING as DB (static `market-data.ts` / `partner-fleet.ts`; no bucket)	BUILD + DONATE (0A)
Threaded messaging	PARTIAL, co-own only (`llc_messages`)	DONATE + BUILD (2E)
Notifications	STUB (email only; no table)	DONATE (0C)
Per-car availability calendar	MISSING (`asset-calendar.tsx` is display-only)	BUILD (2A/2C)
Daily booking flow	MISSING (lead-gen only)	BUILD (2B/2D) — the biggest gap
Payment rail	BUILT but **wrong model** (fee-only direct charge)	MODIFY → on-platform (3B)

Capability	RYDA today	Action
Configurable fee engine	PARTIAL (percent-only, single-payer)	MODIFY (3A)
Security deposit	MISSING	BUILD (3C)
Reservation agreement / e-sign	co-own only; **Dropbox Sign installed**	BUILD (4B)
Check-in / return	BUILT, co-own only (`vehicle_handovers` , 0034)	MODIFY (4C)

---

7. Progress checklist

Tick as PRs merge; add the PR link + one line each. (Boxes mirror §4.)

- [ ] 0A rental_listings + media bucket + RLS
- [ ] 0B renter / operator-staff roles
- [ ] 0C notifications table + notify()
- [ ] 0D partner_id FK migration
- [ ] 1A renter identity + license/age verification
- [ ] 1B rental signup / onboarding
- [ ] 2A availability + blackout model
- [ ] 2B rental_bookings + EXCLUDE + state-machine trigger
- [ ] 2C availability calendar UI + server quote
- [ ] 2D request-to-book flow
- [ ] 2E renter↔operator messaging
- [ ] 2F operator fleet dashboard
- [ ] 2G renter "my rentals" dashboard
- [ ] 3A configurable fee engine
- [ ] 3B full on-platform rail
- [ ] 3C security-deposit hold
- [ ] 3D refunds / disputes / cancellation
- [ ] 4A payment copy + legal sweep
- [ ] 4B reservation agreement / e-sign
- [ ] 4C check-in / return rewire
- [ ] 4D CSP + images + expiry job

Built from a read-only inventory of `ryda-web-rental` (branch `feat/dt-rental-first-site`, migration head 0043) cross-referenced against the Mainstable/eqmarket marketplace. Verify file paths and line numbers against the live tree before editing — the code is the source of truth.